

手摸手之如何完成列车数据检索

星球里有老板说不清楚列车数据如何完成检索的，这不，它来了。

12306 项目中列车数据检索接口路径 `TicketController#pageListTicketQuery`。

```
40  /**
41   * 车票控制层
42   *
43   * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
44   */
45  @RestController
46  @RequiredArgsConstructor
47  public class TicketController {
48
49      private final TicketService ticketService;
50
51      /**
52       * 根据条件查询车票
53       */
54      @GetMapping("/api/ticket-service/ticket/query")
55      public Result<TicketPageQueryRespDTO> pageListTicketQuery(TicketPageQueryReqDTO requestParam) {
56          return Results.success(ticketService.pageListTicketQuery(requestParam));
57      }
58  }
```

验证数据是否正确

查询列车数据 Service 实现层接口第一行代码，就是通过责任链模式验证数据是否必填以及城市数据是否存在等执行逻辑。

如果对责任链模式不太熟悉的小伙伴，欢迎查看 [手摸手之实现用户购票责任链验证](#) 了解前置知识。

非常简单的一行代码，通过责任链容器调用底层实现。接下来我们看看底层都做了什么逻辑，包含责任链相关的知识就不再赘述。

```
ticketPageQueryAbstractChainContext.handler(TicketChainMarkEnum.TRAIN_QUERY_FILTER.name()
, requestParam);
```

1) 检查相关数据是否为空或空的字符串，这个是最先被执行的。

```

package org.opengoofy.index12306.biz.ticketservice.service.handler.ticket.filter.query;

import cn.hutool.core.util.StrUtil;
import org.opengoofy.index12306.biz.ticketservice.dto.req.TicketPageQueryReqDTO;
import org.opengoofy.index12306.framework.starter.convention.exception.ClientException;
import org.springframework.stereotype.Component;

/**
 * 查询列车车票流程过滤器之验证数据是否为空或空的字符串
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Component
public class TrainTicketQueryParamNotNullChainFilter implements
TrainTicketQueryChainFilter<TicketPageQueryReqDTO> {

    @Override
    public void handler(TicketPageQueryReqDTO requestParam) {
        if (StrUtil.isBlank(requestParam.getFromStation())) {
            throw new ClientException("出发地不能为空");
        }
        if (StrUtil.isBlank(requestParam.getToStation())) {
            throw new ClientException("目的地不能为空");
        }
        if (requestParam.getDepartureDate() == null) {
            throw new ClientException("出发日期不能为空");
        }
    }

    @Override
    public int getOrder() {
        return 0;
    }
}

```

2) 检查数据是否正确。

```

package org.opengoofy.index12306.biz.ticketservice.service.handler.ticket.filter.query;

import cn.hutool.core.collection.ListUtil;

```

```

import com.baomidou.mybatisplus.core.toolkit.Wrappers;
import com.google.common.collect.Maps;
import lombok.RequiredArgsConstructor;
import org.opengoofy.index12306.biz.ticketservice.dao.entity.RegionDO;
import org.opengoofy.index12306.biz.ticketservice.dao.entity.StationDO;
import org.opengoofy.index12306.biz.ticketservice.dao.mapper.RegionMapper;
import org.opengoofy.index12306.biz.ticketservice.dao.mapper.StationMapper;
import org.opengoofy.index12306.biz.ticketservice.dto.req.TicketPageQueryReqDTO;
import org.opengoofy.index12306.framework.starter.cache.DistributedCache;
import org.opengoofy.index12306.framework.starter.convention.exception.ClientException;
import org.redisson.api.RLock;
import org.redisson.api.RedissonClient;
import org.springframework.data.redis.core.HashOperations;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

import java.time.LocalDate;
import java.time.ZoneId;
import java.util.HashMap;
import java.util.List;
import java.util.Objects;

import static
org.opengoofy.index12306.biz.ticketservice.common.constant.RedisKeyConstant.LOCK_QUERY_AL
L_REGION_LIST;
import static
org.opengoofy.index12306.biz.ticketservice.common.constant.RedisKeyConstant.QUERY_ALL_REG
ION_LIST;

/**
 * 查询列车车票流程过滤器之验证数据是否正确
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Component
@RequiredArgsConstructor
public class TrainTicketQueryParamVerifyChainFilter implements
TrainTicketQueryChainFilter<TicketPageQueryReqDTO> {

    private final RegionMapper regionMapper;
    private final StationMapper stationMapper;

```

```

private final DistributedCache distributedCache;
private final RedissonClient redissonClient;

private static boolean FLAG = false;

@Override
public void handler(TicketPageQueryReqDTO requestParam) {
    // 判断出发日期不能小于当前日期，毕竟不可能买上一天的车票，如果是的话，一定是异常请求
    if
(requestParam.getDepartureDate().toInstant().atZone(ZoneId.systemDefault()).toLocalDate()
.isBefore(LocalDate.now())) {
        throw new ClientException("出发日期不能小于当前日期");
    }
    // 验证出发地和目的地是否存在
    StringRedisTemplate stringRedisTemplate = (StringRedisTemplate)
distributedCache.getInstance();
    HashOperations<String, Object, Object> hashOperations =
stringRedisTemplate.opsForHash();
    // 查询出发站点和到达站点是否存在，如果不存在也一样属于异常数据
    List<Object> actualExistList = hashOperations.multiGet(
        QUERY_ALL_REGION_LIST,
        ListUtil.toList(requestParam.getFromStation(),
requestParam.getToStation())
    );
    // 这里有个毕竟坑的地方，就算为空，也会返回数据，所以我们通过 filter 判断对象是否为
空

    long emptyCount = actualExistList.stream().filter(Objects::isNull).count();
    // 如果为空的记录是0的话，就证明出发站点和到达站点存在，正常返回即可
    if (emptyCount == 0L) {
        return;
    }
    // 如果出发站点和到达站点都不存在或者仅存在一个，直接抛出异常
    // FLAG = true 代表已经加载过一次，此时还是空，证明说数据库也没有这两个站点信息，抛
异常

    if ((emptyCount == 2L && FLAG && !distributedCache.hasKey(QUERY_ALL_REGION_LIST))
        || emptyCount == 1L) {
        throw new ClientException("出发地或目的地不存在");
    }
    // 如果FLAG=false代表有可能缓存没有数据，但数据库可能有，此时向下查询数据库
    // 为了避免缓存击穿，所以这里是用了分布式锁
    RLock lock = redissonClient.getLock(LOCK_QUERY_ALL_REGION_LIST);

```

```

lock.lock();
try {
    // 获取完分布式锁，避免重复且无用的加载数据库，通过双重判定锁的形式，在查询一次缓存

    // 如果缓存存在，直接返回即可。
    if (distributedCache.hasKey(QUERY_ALL_REGION_LIST)) {
        actualExistList = hashOperations.multiGet(
            QUERY_ALL_REGION_LIST,
            ListUtil.toList(requestParam.getFromStation(),
requestParam.getToStation())
        );
        emptyCount = actualExistList.stream().filter(Objects::nonNull).count();
        if (emptyCount != 2L) {
            throw new ClientException("出发地或目的地不存在");
        }
        return;
    }
    // 因为站点是可以传城市名称的，也就是 Region，所以我们需要查询站点以及城市两张表
    List<RegionDO> regionDOList =
regionMapper.selectList(Wrappers.emptyWrapper());
    List<StationDO> stationDOList =
stationMapper.selectList(Wrappers.emptyWrapper());
    HashMap<Object, Object> regionValueMap = Maps.newHashMap();
    for (RegionDO each : regionDOList) {
        regionValueMap.put(each.getCode(), each.getName());
    }
    for (StationDO each : stationDOList) {
        regionValueMap.put(each.getCode(), each.getName());
    }
    // 查询完后，通过 putAll 的形式存入缓存，避免多次 put 浪费网络 IO
    hashOperations.putAll(QUERY_ALL_REGION_LIST, regionValueMap);
    // 设置 FLAG = true，代表已经加载过初始化数据
    FLAG = true;
    // 再查询一次，查看是否存在。这里偷懒了，其实我们可以通过 regionValueMap 直接判
断

    actualExistList = hashOperations.multiGet(
        QUERY_ALL_REGION_LIST,
        ListUtil.toList(requestParam.getFromStation(),
requestParam.getToStation())
    );
    // 如果加载后还是为空，那么直接抛出异常

```

```
        emptyCount = actualExistList.stream().filter(Objects::nonNull).count();
        if (emptyCount != 2L) {
            throw new ClientException("出发地或目的地不存在");
        }
    } finally {
        // 分布式锁解锁
        lock.unlock();
    }
}

@Override
public int getOrder() {
    return 10;
}
}
```

加载城市数据

12306 站点查询实际功能中，比如你搜索了北京南到杭州东的搜索条件，它会帮你列出北京到杭州所有的列车车次。这个很好实现，直接通过站点关联到城市，通过城市查询列车即可。

我们在缓存中，有一个 Hash 结构数据，专门负责保存列车站点 Code 值与城市之间的关联关系。

可能很多老板会问，为什么列车站点存的是 Code 值，城市存的是名称？这个是为了方便大家查看，最初写 MySQL 数据库中的记录都是使用城市名称，而不是 Code 值，所以大家知道即可。

Hash

mading0924:index12306-ticket-service:region_train_station_mappir ✓

TTL

-1

×

✓

🗑️

🔄

🔌

添加新行

ID (Total: 11)	Key ⚡	Value ⚡	输入关键字搜索			
1	HNH	海宁	📄	✎	🗑️	🔌
2	HZH	杭州	📄	✎	🗑️	🔌
3	HGH	杭州	📄	✎	🗑️	🔌
4	VNP	北京	📄	✎	🗑️	🔌
5	BJP	北京	📄	✎	🗑️	🔌
6	NKH	南京	📄	✎	🗑️	🔌
7	DZP	德州	📄	✎	🗑️	🔌
8	JXH	嘉兴	📄	✎	🗑️	🔌
9	NJH	南京	📄	✎	🗑️	🔌
10	NGH	宁波	📄	✎	🗑️	🔌
11	JGK	济南	📄	✎	🗑️	🔌

代码加载逻辑如下：

```
// 通过批量查询方式获取出发站点和到达站点对应的城市集合
List<Object> stationDetails = stringRedisTemplate.opsForHash()
    .multiGet(REGION_TRAIN_STATION_MAPPING,
Lists.newArrayList(requestParam.getFromStation(), requestParam.getToStation()));
// 判断为空问题，同上验证查询数据
long count = stationDetails.stream().filter(Objects::isNull).count();
// 如果城市数据为空，代表缓存中没有这个值，需要从数据库加载
if (count > 0) {
    // 避免缓存击穿，通过分布式锁方式解决该问题
    RLock lock = redissonClient.getLock(LOCK_REGION_TRAIN_STATION_MAPPING);
    lock.lock();
    try {
        // 双重判定锁，规避数据库无效请求
        stationDetails = stringRedisTemplate.opsForHash()
            .multiGet(REGION_TRAIN_STATION_MAPPING,
Lists.newArrayList(requestParam.getFromStation(), requestParam.getToStation()));
        count = stationDetails.stream().filter(Objects::isNull).count();
        if (count > 0) {
            // 查询列车站点数据与城市信息
            List<StationDO> stationDOList =
stationMapper.selectList(Wrappers.emptyWrapper());
```

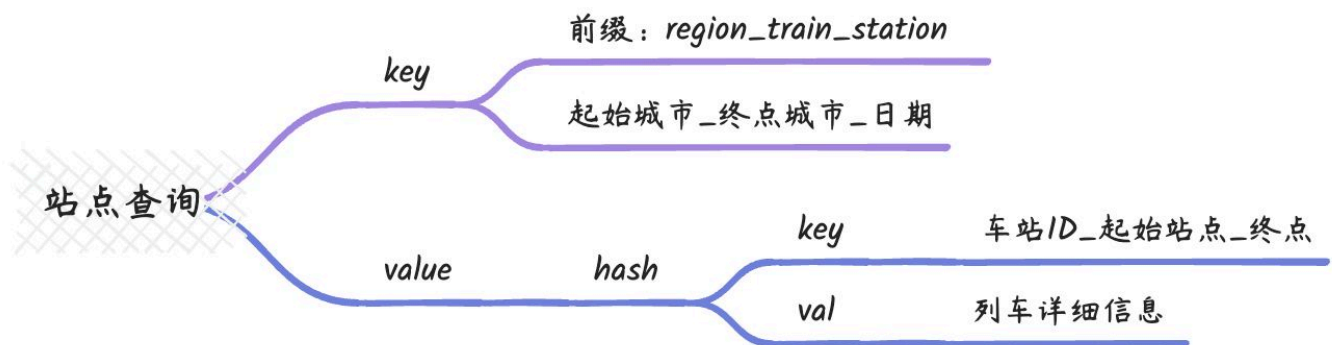
```

        Map<String, String> regionTrainStationMap = new HashMap<>();
        stationDOList.forEach(each -> regionTrainStationMap.put(each.getCode(),
each.getRegionName()));
        // 通过 putAll 批量保存方式存入 Redis, 避免多次 put 网络 IO 消耗
        stringRedisTemplate.opsForHash().putAll(REGION_TRAIN_STATION_MAPPING,
regionTrainStationMap);
        stationDetails = new ArrayList<>();
        stationDetails.add(regionTrainStationMap.get(requestParam.getFromStation()));
        stationDetails.add(regionTrainStationMap.get(requestParam.getToStation()));
    }
} finally {
    lock.unlock();
}
}

```

查询列车站点信息

在 [手摸手之车票搜索为什么用Redis而不是ES?](#) 中说了, 列车站点数据存入 Redis 中, 结构如下:



Redis 中实际数据存储如下:

Hash	mading0924:index12306-ticket-service:region_train_station:北京_杭 ✓	TTL	-1 × ✓			
添加新行						
ID (Total: 4)	Key	Value	输入关键字搜索			
1	1_北京南_杭州东	{ "arrival": "杭州东", "arrivalFlag": false, "arrivalTime": ...	   			
2		{ "arrival": "杭州东", "arrivalFlag": true, "arrivalTime": "23:22", "daysArrived": 0, "departure": "北京南", "departureFlag": true, "departureTime": "19:04", "duration": "04:18", "saleStatus": 0, "saleTime": "05-15 14:30", "trainId": "2", "trainNumber": "G39", "trainTags": ["0", "3"], "trainType": 0 }				
3						
4	2_北京南_杭州东	{ "arrival": "杭州东", "arrivalFlag": true, "arrivalTime": "...	   			

注意，列车详细信息中是没有列车余票数据的，因为这个数据是实时变更的，存入这里无法更新。

查询代码如下所示：

```
List<TicketListDTO> seatResults = new ArrayList<>();
// 构建查询 Redis 中 Hash 结构 Key, Key前缀 + 出发城市 + 到达城市
String buildRegionTrainStationHashKey = String.format(REGION_TRAIN_STATION,
stationDetails.get(0), stationDetails.get(1));
Map<Object, Object> regionTrainStationAllMap =
stringRedisTemplate.opsForHash().entries(buildRegionTrainStationHashKey);
// 如果为空，兜底查询数据库再放入缓存
if (MapUtil.isEmpty(regionTrainStationAllMap)) {
    // 老规矩上分布式锁
    RLock lock = redissonClient.getLock(LOCK_REGION_TRAIN_STATION);
    lock.lock();
    try {
        // 老规矩双重判定锁
        regionTrainStationAllMap =
stringRedisTemplate.opsForHash().entries(buildRegionTrainStationHashKey);
        if (MapUtil.isEmpty(regionTrainStationAllMap)) {
            // 加载数据库列车相关信息，并构建出一趟列车详细记录
            LambdaQueryWrapper<TrainStationRelationDO> queryWrapper =
Wrappers.lambdaQuery(TrainStationRelationDO.class)
                .eq(TrainStationRelationDO::getStartRegion, stationDetails.get(0))
                .eq(TrainStationRelationDO::getEndRegion, stationDetails.get(1));
            List<TrainStationRelationDO> trainStationRelationList =
trainStationRelationMapper.selectList(queryWrapper);
```

```

        for (TrainStationRelationDO each : trainStationRelationList) {
            TrainDO trainDO = distributedCache.safeGet(
                TRAIN_INFO + each.getTrainId(),
                TrainDO.class,
                () -> trainMapper.selectById(each.getTrainId()),
                ADVANCE_TICKET_DAY,
                TimeUnit.DAYS);
            TicketListDTO result = new TicketListDTO();
            result.setTrainId(String.valueOf(trainDO.getId()));
            result.setTrainNumber(trainDO.getTrainNumber());
            result.setDepartureTime(convertDateToLocalTime(each.getDepartureTime(),
"HH:mm"));
            result.setArrivalTime(convertDateToLocalTime(each.getArrivalTime(),
"HH:mm"));

            result.setDuration(DateUtil.calculateHourDifference(each.getDepartureTime(),
each.getArrivalTime()));
            result.setDeparture(each.getDeparture());
            result.setArrival(each.getArrival());
            result.setDepartureFlag(each.getDepartureFlag());
            result.setArrivalFlag(each.getArrivalFlag());
            result.setTrainType(trainDO.getTrainType());
            result.setTrainBrand(trainDO.getTrainBrand());
            if (StrUtil.isNotBlank(trainDO.getTrainTag())) {
                result.setTrainTags(StrUtil.split(trainDO.getTrainTag(), ","));
            }
            long betweenDay =
cn.hutool.core.date.DateUtil.betweenDay(each.getDepartureTime(), each.getArrivalTime(),
false);

            result.setDaysArrived((int) betweenDay);
            result.setSaleStatus(new Date().after(trainDO.getSaleTime()) ? 0 : 1);
            result.setSaleTime(convertDateToLocalTime(trainDO.getSaleTime(), "MM-dd
HH:mm"));

            seatResults.add(result);

            regionTrainStationAllMap.put(CacheUtil.buildKey(String.valueOf(each.getTrainId()),
each.getDeparture(), each.getArrival()), JSON.toJSONString(result));
        }
        // 全部加载完，批量保存到 Redis Hash 结构中
        stringRedisTemplate.opsForHash().putAll(buildRegionTrainStationHashKey,
regionTrainStationAllMap);

```

```

    }
} finally {
    lock.unlock();
}
}

```

查询出来列车基本信息后，开始对列车按照出发时间进行排序。

```

seatResults = CollUtil.isEmpty(seatResults)
    ? regionTrainStationAllMap.values().stream().map(each ->
JSON.parseObject(each.toString(), TicketListDTO.class)).toList()
    : seatResults;
seatResults = seatResults.stream().sorted(new TimeStringComparator()).toList();

```

其中定义了自定义时间比较器，就是将时间转换为时分方式进行比较，时间早的查询展示在上方。

```

package org.opengoofy.index12306.biz.ticketservice.toolkit;

import org.opengoofy.index12306.biz.ticketservice.dto.domain.TicketListDTO;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Comparator;

/**
 * 自定义时间比较器
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public class TimeStringComparator implements Comparator<TicketListDTO> {

    private static final DateTimeFormatter FORMATTER =
DateTimeFormatter.ofPattern("HH:mm");

    @Override
    public int compare(TicketListDTO ticketList1, TicketListDTO ticketList2) {
        LocalDateTime localTime1 = LocalDateTime.parse(ticketList1.getDepartureTime(),
FORMATTER);
        LocalDateTime localTime2 = LocalDateTime.parse(ticketList2.getDepartureTime(),

```

```
FORMATTER);
        return localTime1.compareTo(localTime2);
    }
}
```

查询列车余票信息

列车基本信息已经全部填充完毕了，接下来就是查询列车余票信息并填充到基本信息中。

上面讲过，列车余票数据是实时变更的，如果在存储到基本信息中，就没办法变更了，所以单独存储。

```
for (TicketListDTO each : seatResults) {
    // 加载列车对应的座位价格数据
    // safeGet 就是安全获取缓存方案，底层加了分布式锁和双重判定锁
    // 如果查询 TRAIN_STATION_PRICE 数据为空，则加载数据库，并放入缓存
    String trainStationPriceStr = distributedCache.safeGet(
        String.format(TRAIN_STATION_PRICE, each.getTrainId(), each.getDeparture(),
            each.getArrival()),
        String.class,
        () -> {
            LambdaQueryWrapper<TrainStationPriceDO> trainStationPriceQueryWrapper =
                Wrappers.lambdaQuery(TrainStationPriceDO.class)
                    .eq(TrainStationPriceDO::getDeparture, each.getDeparture())
                    .eq(TrainStationPriceDO::getArrival, each.getArrival())
                    .eq(TrainStationPriceDO::getTrainId, each.getTrainId());
            return
                JSON.toJSONString(trainStationPriceMapper.selectList(trainStationPriceQueryWrapper));
        },
        ADVANCE_TICKET_DAY,
        TimeUnit.DAYS
    );
    List<TrainStationPriceDO> trainStationPriceDOList =
        JSON.parseArray(trainStationPriceStr, TrainStationPriceDO.class);
    List<SeatClassDTO> seatClassList = new ArrayList<>();
    // 循环遍历座位价格数据，获取到座位对应的余票，并最终放入到列车基本信息中
    trainStationPriceDOList.forEach(item -> {
        String seatType = String.valueOf(item.getSeatType());
        String keySuffix = StrUtil.join("_", each.getTrainId(), item.getDeparture(),
```

```

item.getArrival());
    Object quantityObj =
stringRedisTemplate.opsForHash().get(TRAIN_STATION_REMAINING_TICKET + keySuffix,
seatType);
    int quantity = Optional.ofNullable(quantityObj)
        .map(Object::toString)
        .map(Integer::parseInt)
        .orElseGet(() -> {
            Map<String, String> seatMarginMap =
seatMarginCacheLoader.load(String.valueOf(each.getTrainId()), seatType,
item.getDeparture(), item.getArrival());
            return
Optional.ofNullable(seatMarginMap.get(String.valueOf(item.getSeatType()))).map(Integer::p
araseInt).orElse(0);
        });
    seatClassList.add(new SeatClassDTO(item.getSeatType(), quantity, new
BigDecimal(item.getPrice()).divide(new BigDecimal("100"), 1, RoundingMode.HALF_UP),
false));
    });
    each.setSeatClassList(seatClassList);
}

```

构建列车返回数据

查看 12306 列车查询页可知，会存在不同的查询条件，这些查询条件都是通过本次查询所有列车数据构建出来的。大家可以尝试下，不同地区的不同查询列车数据，车次类型、出发车站、到达车站以及车次席别都不同。

首页

车票

团购服务

会员服务

站车服务

商旅服务

出行指南

信息查询

☒ 单程
 ☐ 往返

出发地
 目的地
 出发日
 返程日

☒ 普通
 ☐ 学生

09-03 09-04 09-05 09-06 09-07 09-08 09-09 09-10 09-11 09-12 09-13 09-14 09-15 09-16 **09-17 周日**

车次类型: ☒ 全部 ☐ GC-高铁/城际 ☐ D-动车 ☐ Z-直达 ☐ T-特快 ☐ K-快速 ☐ 其他 ☐ 复兴号 ☒ 智能动车组

发车时间:

出发车站: ☒ 全部 ☐ 北京丰台 ☐ 北京南 ☐ 北京

到达车站: ☒ 全部 ☐ 天津 ☐ 滨海 ☐ 武清 ☐ 塘沽 ☐ 天津南 ☐ 天津西 ☐ 军粮城北 ☐ 北辰

车次席别: ☒ 全部 ☐ 商务座 ☐ 一等座 ☐ 二等座 ☐ 高级软卧 ☐ 一等卧 ☐ 二等卧 ☐ 软卧 ☐ 硬卧 ☐ 硬座

北京北 --> 天津 (9月17日 周日) 共计4个车次 您可使用中转换乘功能, 查询途中换乘一次的部分列车余票情况。 ☐ 显示积分兑换车次 ☐ 显示全部可预订车次

车次	出发站 到达站	出发时间 到达时间	历时	商务座 特等座	一等座	二等座 二等包座	高级 软卧	软卧 一等卧	动卧	硬卧 二等卧	软座	硬座	无座	其他	备注
G45 智复	北京南 天津南	06:54 07:25	00:31 当日到达	1	候补	有	--	--	--	--	--	--	--	--	预订
G1 智复	北京南 天津南	07:00 07:31	00:31 当日到达	候补	18	有	--	--	--	--	--	--	--	--	预订
G1099 智复	北京南 天津南	07:30 08:04	00:34 当日到达	候补	候补	11	--	--	--	--	--	--	--	--	预订
G11 智复	北京南 天津南	12:00 12:31	00:31 当日到达	候补	2	9	--	--	--	--	--	--	--	--	预订

如果查询结果中没有满足需求的车次, 您还可以使用中转换乘功能, 查询途中换乘一次的部分列车余票情况。
显示的卧铺票价均为上铺票价, 供您参考。具体票价以您确认支付时实际购买的铺别票价为准。如因运力原因或其他不可控因素导致列车调度调整时, 当前车型可能会发生变动。

北京北 — — 天津 [查询更多中转方案](#)

接下来就是通过构建者模式构建列车查询返回数据。

```
return TicketPageQueryRespDTO.builder()
    .trainList(seatResults)
    .departureStationList(buildDepartureStationList(seatResults))
    .arrivalStationList(buildArrivalStationList(seatResults))
    .trainBrandList(buildTrainBrandList(seatResults))
    .seatClassTypeList(buildSeatClassList(seatResults))
    .build();
```

文末总结

大家在看过整体代码后, 不知道有没有一个疑问, 是不是请求 Redis 太多了呀。目前代码处于 v1 版本, 后续会优化 Redis 调用链路, 同时推出 v2 版本查询接口, 敬请期待。

更新: 2023-09-13 14:19:40

原文: <<https://www.yuque.com/magestack/12306/tygc8hs113al2c2z>>